Cloud Computing - Unit 1

Fundamentals of Cloud Computing | MCA Sem III | Easy Exam Notes

1. What is Cloud Computing?

- **Computing** = using hardware + software to solve any goal-oriented task (build systems, find information, make systems smart using communication).
- **Cloud computing** = getting **storage, networking, servers and software (IT infrastructure)** over the **internet** and paying only for what you use.
- Think of it like a **utility**: electricity, water, telephone. You do not build a power plant at home - you just switch on and pay the bill. Same with cloud: no buying servers, just use and pay (**pay-per-use**).
- Keywords: **ubiquitous** (everywhere), **convenient**, **on-demand**, **pay-per-use**.
- **1969 - Leonard Kleinrock** (ARPANET scientist) predicted 'computer utilities' serving homes and offices like electricity and telephone. Cloud makes this real.
- Typical user view: 'I don't care where my servers are or who manages them. I want them always available from any device and I will pay as long as I need.'

2. Definitions (very important!)

NIST definition

Cloud computing is a model for easy, on-demand network access to a **shared pool of configurable computing resources** (networks, servers, storage, applications, services) that can be **rapidly provisioned and released** with **minimal management effort**.

Armbrust et al.

Cloud computing = **applications delivered as services over the Internet** + the **hardware and system software in data centers** that provide those services.

Buyya et al.

A cloud is a **parallel and distributed system** of interconnected, **virtualized** computers that are **dynamically provisioned** and shown as one or more unified resources, based on **SLAs** (service-level agreements) agreed between provider and consumer.

- Exam tip - remember one keyword per definition: NIST = **shared pool + on-demand**, Armbrust = **apps + data centers**, Buyya = **virtualized + SLA**.

3. Vision of Cloud Computing

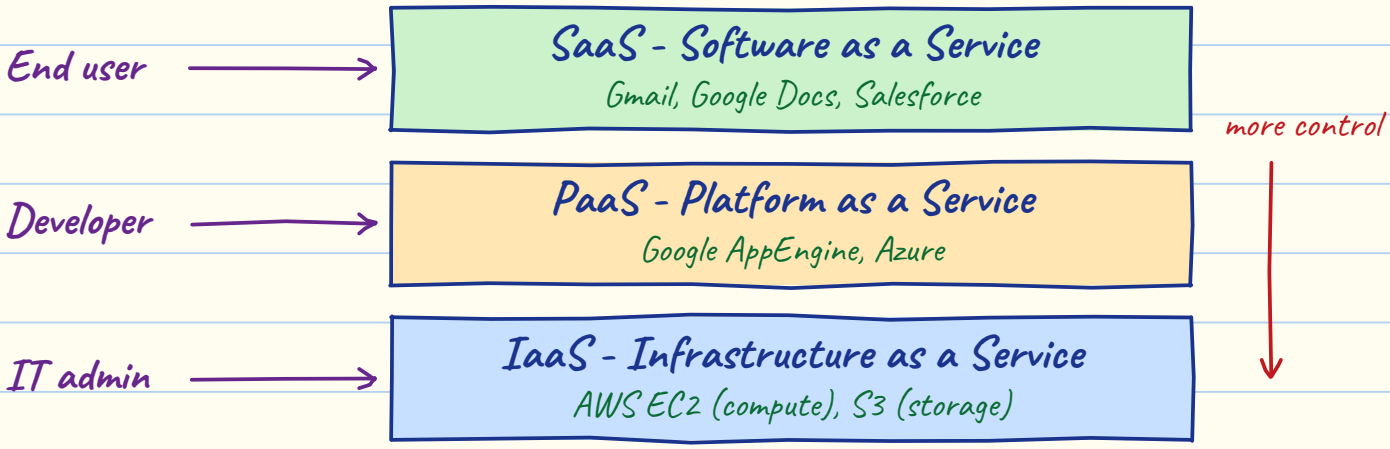
- Long-term vision: **IT services are traded as utilities in an open market**, without technological and legal barriers.
- Using cloud is now as easy as a **credit-card payment** on the internet.

A closer look - who benefits?

- **Large companies** offload some work to the cloud (e.g. Amazon EC2 and S3).
- **Small companies / start-ups** turn ideas into business quickly, **no big up-front cost**.
- **Developers** focus on business logic, not on infrastructure and scalability headaches.
- **End users** open their documents anywhere, from any device.
- Big change: we now think of IT as a **utility**, not as a product we must own.

4. Cloud Reference Model

Cloud gives 3 types of services (the 'layers'):



- **IaaS** - you rent virtual compute, storage, networking (like renting an empty plot).
- **PaaS** - you get a ready platform / runtime to build and run your apps (like renting a plot with a building frame).
- **SaaS** - you simply use finished software through the browser (like staying in a furnished flat).
- **Technologies behind cloud:** Web 2.0, Service Orientation, Virtualization.

5. Cloud in the Industry

- **Big players:** Google Cloud, Amazon Web Services (AWS), Microsoft Azure, IBM Cloud, Alibaba Cloud.
- **Careers:** Cloud Solution Architect, Cloud Developer / Software Engineer, Cloud DevOps Engineer, Cloud System Engineer / Administrator, SysOps Administrator, Cloud Product Manager, Cloud Consultant.
- **Certifications:** AWS Solutions Architect - Associate, Azure Fundamentals, Google Associate Cloud Engineer, IBM Cloud Foundations, CCSK (Cloud Security Alliance).

6. Historical Developments

5 core technologies that made cloud computing possible:

- 1. Distributed Systems 2. Virtualization 3. Web 2.0 4. Service Orientation 5. Utility Computing

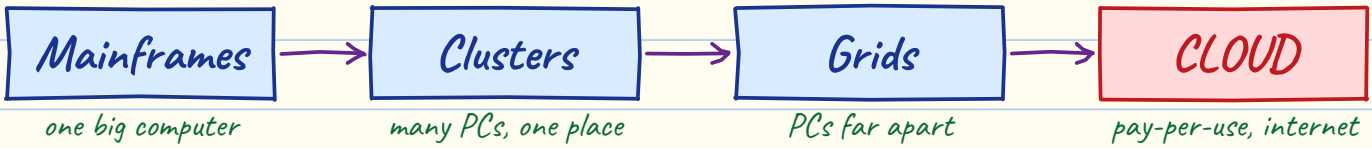
6.1 Distributed Systems

Definition

A distributed system has **multiple components on different machines** that communicate and coordinate their actions so that it looks like **one single system** to the user.

- Two key points: (i) many independent components but users see **one entity**; (ii) main purpose = **share resources and use them better**.
- Examples: telephone and cellular networks, **Peer-to-Peer (P2P)** networks.
- Benefits: **horizontal scalability, reliability, performance**.

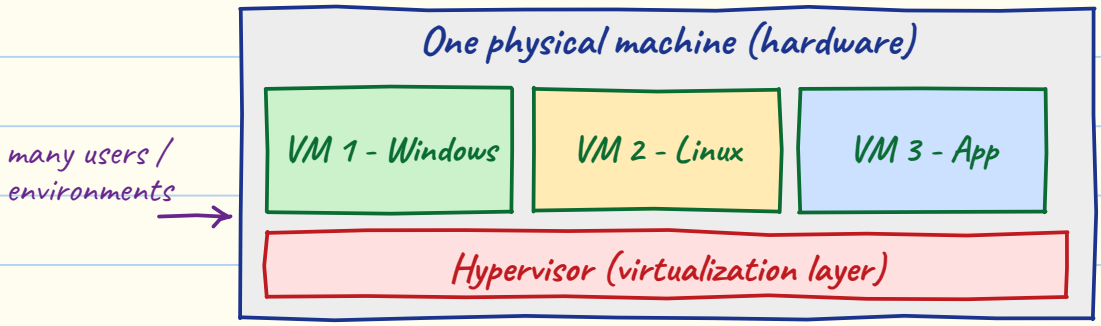
Three milestones that led to cloud:



- **Mainframes** - one very powerful central computer used by many people.
- **Clusters** - a group of normal computers connected in one place, working as one.
- **Grids** - networked computers (even far apart) working together like a **virtual supercomputer** for huge jobs such as data analysis or weather modeling.

6.2 Virtualization

- Technology that lets you create useful IT services from resources that are normally **tied to hardware**.
- Lets you use a machine's **full capacity** by sharing it among many users or environments.
- How it works: a **hypervisor** sits on the hardware and splits it into many **virtual machines (VMs)**, each acts like a separate computer.



6.3 Web 2.0

- **Second generation of the World Wide Web** - from static HTML pages to **interactive, dynamic** pages.
- People can **collaborate and share** online: social media, blogs, web communities. Anyone can contribute, no technical skill needed.
- Makes internet applications feel like **local applications**.
- Examples: **Google Docs, Google Maps, Flickr, Facebook, Twitter, YouTube, Blogger, Wikipedia**.

6.4 Service-Oriented Computing (SOC)

- Paradigm that uses **services as the basic building blocks** of applications. Gives rapid, low-cost, flexible, interoperable, evolvable systems.
- A service must be **loosely coupled, reusable, programming-language independent, location transparent**.

SOC gave 3 important ideas to cloud:

- **Quality of Service (QoS)** - set of attributes to judge a service: response time, security, transactional integrity, reliability, scalability, availability.
- **Software-as-a-Service (SaaS)** - idea inherited from **ASPs (Application Service Providers)** who deliver software from a central datacenter over the network on **subscription / rental**.
- **Web Services** - a standard way for client and server applications on the WWW to talk to each other, even if built in different languages.

Web service protocols

- **SOAP** (Simple Object Access Protocol) - common 'middle language' so apps in different languages can talk easily.
- **WSDL** (Web Services Description Language) - an **XML file** that tells the client what the service does and how to connect to it.
- **REST** (REpresentational State Transfer) - builds web services that are **lightweight, maintainable, scalable**.

6.5 Utility-Oriented Computing

Definition

Utility computing is a **service-provisioning model** where resources like **storage, compute power, applications and infrastructure** are packaged and offered on a **pay-per-use** basis.

7. Building Cloud Environments

- Two parts: (a) building **applications** that use the cloud, (b) building the **frameworks, platforms and infrastructure** that deliver cloud services.

Application development

- Cloud apps can **scale up or down on demand**. Types: **Web applications** and **resource-intensive applications** (data-intensive / compute-intensive).

Cloud gives dynamic scaling by:

- (a) **renting** compute power, storage and networking;
- (b) giving **runtime environments** made for scalability and dynamic sizing;
- (c) giving **application services** that act like desktop apps but are fully hosted and managed by the provider.

8. Characteristics and Benefits

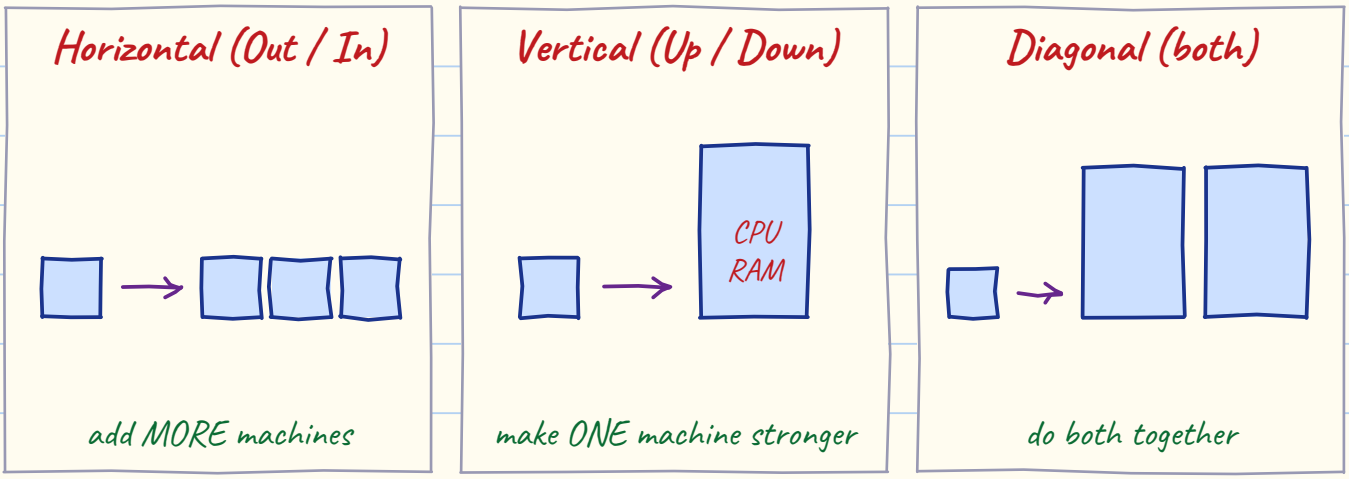
Benefits for both **consumers (CSC)** and **providers (CSP)**:

- **No up-front commitment** - no need to buy hardware first.
- **On-demand access** - get resources when you need them.
- **Nice pricing** - pay-per-use, low cost.
- **Simplified application acceleration and scalability** - apps grow easily.
- **Efficient resource allocation** - resources shared and used well.
- **Energy efficiency** - fewer idle machines, less power wasted.
- **Seamless creation and use of third-party services** - plug in other services easily.

9. Scalability

Definition

Scalability = the ability to **handle growing or shrinking resources** to meet business demand in a capable way.



- **Horizontal scaling (scale out / in)** - add or remove **machines**.
- **Vertical scaling (scale up / down)** - add or remove **power** (CPU, RAM) in the same machine.
- **Diagonal scaling** - **horizontal + vertical** together.

10. Computing Platforms and Technologies

Cloud services are **automated** (no human needed to deliver them), so we need suitable platforms and frameworks to develop cloud apps.

Amazon Web Services (AWS) - IaaS

- Gives virtual **compute, storage and networking**. Famous for on-demand services:
  - **EC2** (Elastic Compute Cloud) = compute, **S3** (Simple Storage Service) = storage.

Google AppEngine - PaaS

- Scalable **runtime environment for web applications**. Build and test on your own PC using the **AppEngine SDK**.
- Services: in-memory caching, scalable data store, job queues, messaging. Languages: **Python, Java, Go**.

Microsoft Azure

- A **cloud operating system** and a platform to build cloud apps.
- Apps are organised by **roles**: **Web role, Worker role, Virtual Machine role**.
- Extra services: storage (relational data and blobs), networking, caching, content delivery.

Hadoop

- **Apache Hadoop** = open-source framework to process **large data sets on commodity hardware**.
- It is an implementation of **MapReduce** (programming model from Google). Developer only gives input data + **map** and **reduce** functions; Hadoop provides the runtime.
- Original aim: large-scale search and text processing on huge web data (stored in BigTable / GFS), using **tens of thousands of processors** in parallel.

Force.com and Salesforce.com

- **Force.com** = cloud platform to develop **social enterprise applications** (data layout, business rules, workflows, user interface).
- **Salesforce.com** = **SaaS** solution for **Customer Relationship Management (CRM)**.

Manjrasoft Aneka

- Cloud **application platform** for rapid creation of **scalable applications** and deploying them on various clouds in a seamless, **elastic** way.
- Developers can pick abstractions: **tasks, distributed threads, map-reduce**.

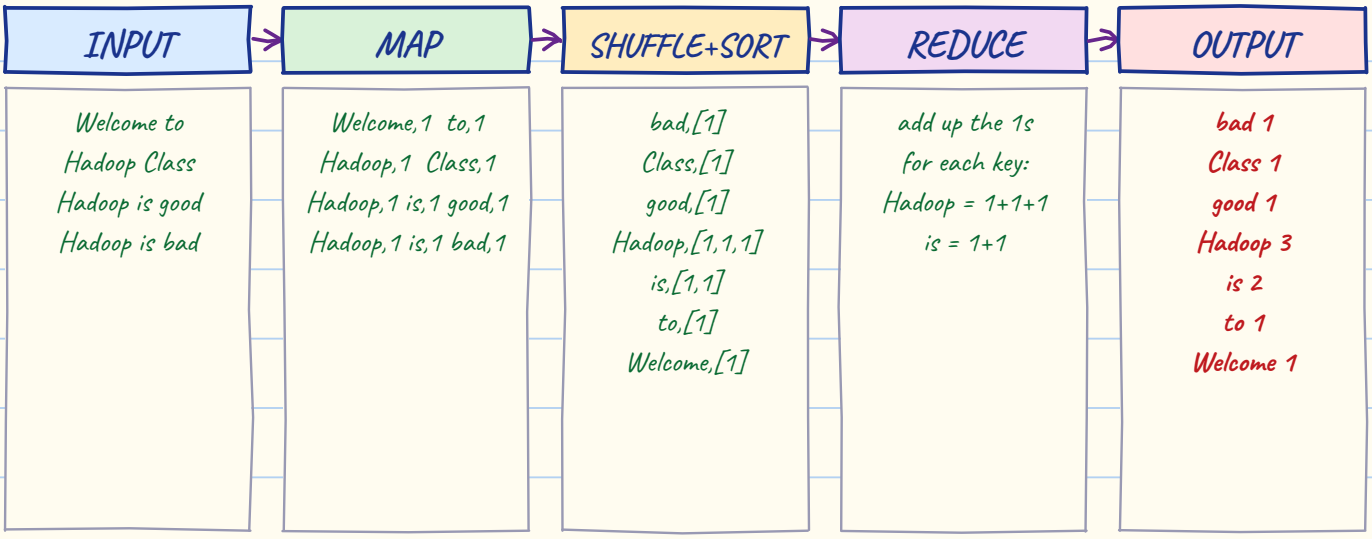
11. MapReduce Programming Model

- Developed at **Google**. Used for **large-scale search and text processing**; processes huge data with **massively parallel computation**.
- **Fault tolerant** - the work keeps going even if processors or networks fail.

- **Hadoop** = open-source implementation (developed at Yahoo); available as pre-packaged AMIs on **Amazon EC2**.
- Works in **2 phases**: **Map** and **Reduce**. Input and output of each phase = **key-value pairs**. Programmer writes the **map function** and **reduce function**.
- **Map** transforms the input data; **Reduce** combines / aggregates the map output.

Word-count example

Input: 'Welcome to Hadoop Class', 'Hadoop is good', 'Hadoop is bad'



Final output: bad 1, Class 1, good 1, **Hadoop 3**, **is 2**, to 1, Welcome 1

12. Quick Revision - Likely Questions

2-mark questions

Define cloud computing (NIST). | What is utility computing? | Name any 3 cloud providers. | What is a hypervisor? | Expand SOAP, WSDL, REST. | What is scalability? | What is a grid?

5-mark questions

Explain characteristics of cloud computing. | Types of scaling with diagram. | Explain Web 2.0 with examples. | Explain MapReduce with word-count example. | Write notes on AWS / AppEngine / Azure.

10-mark questions

Explain the 5 core technologies behind cloud computing. | Explain cloud reference model (IaaS, PaaS, SaaS) and benefits. | Explain distributed systems: mainframes, clusters, grids. | Discuss service-oriented computing, QoS and web services.